Project 12 (Bonus) — MCPResearch-Multi-Agent-Pipeline-Exposed-As-One-MCP-Tool

Type: MCP Server (1 Tool, wrapping a 2-agent pipeline) · **Stack:** Python, `mcp` SDK (MCPServer), `langchain-openai`, asyncio, Pydantic · **Level:** Advanced

Tagline: A Researcher→Fact-Checker revision pipeline, exposed to any MCP client as one plain `deep_research(topic)` call — with a progress trace and a timeout/partial fallback so it always returns *something*.

> Not part of the numbered Doc01-19 arc — a bonus/portfolio project built on 06_tools_function_calling's MCP sections and 11_multi_agent_systems's generator→critic pattern. Builds on Project 7 (MCP server) and Project 4 (Writer/Reviewer loop).

Charter (what this project is)

Wrap a whole agentic pipeline behind **one** MCP tool — the "agent-as-a-service" pattern. A Researcher drafts a summary, a Fact-Checker rejects it with specific issues, the Researcher redrafts using that feedback, bounded by a hard round cap. All of that machinery is hidden behind a single `deep_research(topic) -> str` tool any MCP client can call, with no idea of the loop underneath. The design problem is deciding what a slow, multi-step process should *look like from the outside*: a trace so the caller isn't staring at a black box, and a timeout + partial fallback so a slow or broken run returns a marked partial answer instead of hanging forever.

Jump to: Project Structure · Setup · How testing works · Step 1 · Step 2 · Step 3 · Step 4

The Story — what you're actually building

A teammate wants "a fact-checked summary of X" from their own tool, without building a research agent themselves. You give them one function: `deep_research("Retrieval-Augmented Generation (RAG)")`. Underneath, a Researcher drafts, a Fact-Checker catches that RAG was *not* "introduced by OpenAI in 2019" (it's Lewis et al., Meta AI, 2020), the Researcher fixes exactly that, and the Fact-Checker approves — but the caller sees none of that machinery, just a clean, checked answer.

Step 1 proves the two-agent pipeline works as a plain Python program, no MCP anywhere. Step 2 wraps it in the thinnest possible MCP tool. Step 3 fixes the problem that creates — a caller waiting on a slow, multi-round call with zero feedback — by returning a short revision *trace* alongside the

answer. Step 4 fixes the other real problem — a run that's too slow or breaks — with a timeout and a "best draft so far" fallback, so the tool always returns something.

> **Before you read further — think about it yourself:** which layer owns the timeout — the Researcher, or the whole-pipeline MCP call? And the pipeline is a *synchronous* multi-round loop; the MCP tool is `async`. If you wrap a plain blocking call in `asyncio.wait_for`, does it actually time out, or does it block the event loop anyway? Settle both before Step 4.

Where This Fits

A bonus/portfolio project for the "agent-as-a-service" pattern: a whole workflow offered as one callable capability over a standard protocol, so other teams' tools can use it without depending on its internal steps. It's a harder skill than building a simple MCP server (Project 7) or an agent that calls one (Project 8) — here *you* decide what a slow, multi-step process looks like from the outside.

Project Structure (the professional layout)

`mcp_server.py` and `main.py` at the repo root; all pipeline code in `src/`.

The final layout (after Step 4)

```
<repo folder>/   (MCPResearch-Multi-Agent-Pipeline-Exposed-As-One-MCP-Tool/)
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── fact_checker.py     FactCheckVerdict, run_fact_checker()
│   │   └── researcher.py       Draft, run_researcher()
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py              get_model()
│   ├── prompts/
│   │   ├── fact_checker.txt
│   │   └── researcher.txt
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── logger.py           get_logger() — logs to stderr
│   │   └── prompts.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   ├── formatting.py           format_result(), format_partial()
│   └── pipeline.py             run_pipeline(), PipelineResult, Progress
```

```
├── tests/
│   ├── integration/
│   │   └── test_live_tool.py   real model via the MCP tool
│   └── unit/
│       ├── test_config.py
│       ├── test_formatting.py
│       └── test_pipeline.py    loop + cap, with a fake model
├── .env.example
├── .gitignore
├── LICENSE
├── main.py                     runs the pipeline standalone
├── mcp_server.py               the deep_research MCP tool
├── README.md
└── requirements.txt
```

How the structure grows, step by step

- **Step 1** builds config/exceptions/logger/prompts/llm, `agents/researcher`, `agents/fact_checker`, `pipeline.py`, `main.py`, `test_config`, `test_pipeline`.
- **Step 2** adds `mcp_server.py` with a `deep_research` tool whose body just calls `run_pipeline` and returns the final summary.
- **Step 3** adds `formatting.py` (the revision trace) and a `Progress` holder; the tool returns the trace + summary.
- **Step 4** adds the timeout + partial fallback (`asyncio.to_thread` + `asyncio.wait_for` + `format_partial`) in `mcp_server.py`.

Setup (do this once, before Step 1)

```
cd MCPResearch-Multi-Agent-Pipeline-Exposed-As-One-MCP-Tool
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install "mcp[cli]" langchain-openai python-dotenv pydantic
pip freeze > requirements.txt
```

`.env.example` (copy to `.env`): `OPENAI_API_KEY=sk-...`, `LOG_LEVEL=INFO`, `MAX_REVISION_ROUNDS=3`, `DEEP_RESEARCH_TIMEOUT_SECONDS=60`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`. Every folder under `src/` gets an empty `__init__.py`. `mcp[cli]` installs the MCP Inspector (`mcp dev mcp_server.py`) so you can call `deep_research` by hand.

A Real Example (so this isn't just theory)

Topic: `"Retrieval-Augmented Generation (RAG)"` — round 1 says "introduced by OpenAI in 2019"; the Fact-Checker rejects it; round 2 fixes it to "Lewis et al., Meta AI, 2020" and is approved. `deep_research` returns (from Step 3):

```
Deep research on: Retrieval-Augmented Generation (RAG)

Revision trace:
  Round 1: rejected
    - RAG was introduced by Lewis et al. at Meta AI in 2020, not OpenAI in 2019.
  Round 2: approved

Status: approved after 2 round(s).

Summary:
Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. at Meta AI
(then Facebook AI Research) in 2020 ...
```

Also test: a topic with nothing checkable (`"my opinion of the color blue"`) — proves the round cap matters; and a deliberately slow topic — proves Step 4's timeout/partial path.

How testing works in this project

Two layers, one style (plain OK/WRONG scripts; Doc06/11 don't teach `pytest`):

- **`tests/unit/`** — no model: `test_config` ; `test_formatting` (the trace output on a hand-made result); `test_pipeline` (the revise loop and the round cap, with a tiny **fake model** — no key, deterministic).
- **`tests/integration/test_live_tool.py`** — the real model through the MCP tool: a topic that converges returns an approved trace; and a forced-slow run returns a marked PARTIAL.

Each step ends with a **Tests for this step** box.

Step 1 — The Two-Agent Pipeline, Running Standalone (No MCP Yet)

What this step does: the Researcher, the Fact-Checker, and the loop between them, as a plain Python program — no MCP anywhere.

Why this step matters: if the pipeline's own logic is broken, wrapping it in MCP would hide the bug behind a protocol. Prove it alone first.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write config/exceptions/ `utils/logger` / `utils/prompts` , `models/llm` . 2. Write `agents/researcher.py` : a `Draft` (topic, summary) and `run_researcher(model, topic,`

`feedback=None)`. With `feedback`, the prompt fixes exactly the named issues, not a rewrite. 3. Write `agents/fact_checker.py`: a `FactCheckVerdict` (`approved`, `issues`) and `run_fact_checker(model, draft)` with **real** criteria (wrong dates, invented facts, overconfident claims) — a vague critic approves everything. 4. Write `pipeline.py`: `run_pipeline(model, topic, max_rounds, progress=None) -> PipelineResult` (final_summary, approved, rounds). Loop: draft → check → approved? stop; else feed issues back as feedback, up to `max_rounds`, then stop regardless. 5. Write `main.py` to run the pipeline on 2–3 topics (including a hard one) and print every round.

Your files after Step 1: config, exceptions, `utils/logger`, `utils/prompts`, `models/llm`, `agents/researcher`, `agents/fact_checker`, `pipeline`, `main.py`, `tests/unit/test_config.py`, `tests/unit/test_pipeline.py`.

Tests for this step: `test_config`; `test_pipeline` (fake model) — a wrong first draft is rejected then approved on round 2; a never-approved topic stops at the cap.

Step 2 — Wrap It as a Single MCP Tool

What this step does: `mcp_server.py` exposes `deep_research(topic) -> str` as one `MCPServer` tool whose body just calls `run_pipeline` and returns the final summary.

Why this step matters: the whole point — a multi-agent system reachable through one plain function call over a standard protocol.

What's new vs. Step 1: `mcp_server.py`. **What stays the same:** the pipeline.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `mcp_server.py`: `MCPServer("deep-research-service")`, one `deep_research(topic: str) -> str` tool whose body calls `run_pipeline(...)` and returns `result.final_summary`. Log to **stderr** only (stdout is the protocol channel). 2. Test with `mcp dev mcp_server.py` — call `deep_research` and confirm you get the plain summary (no trace yet).

Your files after Step 2: + `mcp_server.py`.

Tests for this step: the integration test (real tool) returns a non-empty summary.

Step 3 — Report Progress as a Trace

What this step does: returns a short revision **trace** alongside the summary, so the caller isn't waiting on a black box.

Why this step matters: a slow multi-round call with zero feedback is a bad tool. The trace tells the caller what happened each round — without breaking the "one tool call" illusion.

What's new vs. Step 2: `formatting.py` and a `Progress` holder. **What stays the same:** the pipeline's logic.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `formatting.py`: `format_result(result)` → the "Deep research on / Revision trace / Status / Summary" block, and `format_partial(topic, progress)` (used in Step 4). 2. Have `deep_research` return `format_result(run_pipeline(...))`. Pass a `Progress` holder into `run_pipeline` so it records the latest draft and each round (needed in Step 4).

Your files after Step 3: + `formatting.py`, `mcp_server.py` (returns the formatted trace).

Tests for this step: `test_formatting` — the trace output names each round's approved/rejected status and includes the summary.

Step 4 — A Timeout and a Partial-Result Fallback

What this step does: a run that's too slow or breaks returns a clearly-marked partial answer (the best draft so far) instead of hanging.

Why this step matters: a tool that can hang forever isn't one anyone will depend on. It must always return *something*.

What's new vs. Step 3: `asyncio.to_thread` + `asyncio.wait_for` + `format_partial` in `mcp_server.py`. **What stays the same:** the pipeline.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. In `deep_research`, run the pipeline with `await asyncio.wait_for(asyncio.to_thread(run_pipeline, ...), timeout=...)`. **The `to_thread` matters:** the pipeline is synchronous, so wrapping it directly in `wait_for` would block the event loop — run it in a thread first. 2. On `asyncio.TimeoutError`, return `format_partial(topic, progress)` — the best draft recorded so far, clearly marked PARTIAL. (Note: `to_thread` can't cancel the thread, so it keeps running in the background; that's a known limitation worth stating.) 3. Test with a deliberately slow topic / a short timeout — confirm a PARTIAL comes back, not a hang.

Your files after Step 4 (final): the full layout above.

Tests for this step: the integration test's forced-slow case returns a PARTIAL.

Final Deliverable: a one-tool MCP service, `deep_research`, that runs a fact-checked Researcher/Fact-Checker pipeline capped at a fixed number of rounds, reports its progress as a trace, and falls back to a marked partial answer on timeout — a whole agentic workflow behind one callable capability.

Run, Test and Configure the Final Project

```
python main.py                        # the pipeline standalone
mcp dev mcp_server.py                 # call deep_research by hand
python -m tests.unit.test_config
python -m tests.unit.test_formatting
python -m tests.unit.test_pipeline
python -m tests.integration.test_live_tool
```

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	Log verbosity (to stderr)
`MAX_REVISION_ROUNDS`	no	`3`	Hard cap on revise rounds
`DEEP_RESEARCH_TIMEOUT_SECONDS`	no	`60`	Timeout before the partial fallback

Checklist Before You Call This Done

- [ ] The pipeline was proven standalone before any MCP wrapped it
- [ ] `deep_research` is one tool whose body just calls the pipeline
- [ ] The result includes a revision trace, not only the bare summary
- [ ] The revise loop stops at `MAX_REVISION_ROUNDS` on a never-approved topic
- [ ] A slow run returns a marked PARTIAL, not a hang (`to_thread` + `wait_for`)
- [ ] All logging goes to stderr; nothing prints to stdout

Publish It to GitHub

Name the repo `MCPResearch-Multi-Agent-Pipeline-Exposed-As-One-MCP-Tool`, add the `README.md`, commit `.env.example`, push. It shows the agent-as-a-service pattern: a whole workflow behind one standard-protocol call, with a trace and a timeout fallback.

Status

Not started.

Stuck? Ask for **Hint 1** or **Hint 2**. Ask for the full code only if you say **"Show me the solution."**